Commands I ran: database, Django and AWS setup

Session of 27 Oct 2026 · Windows 11 · PowerShell · PostgreSQL 18 on port 6900

Passwords and keys are shown as placeholders. Replace them with your own and never commit them. Run each command one at a time.

1. Make the ESP32 reach the server on your laptop

Allow port 8000 through the Windows firewall (run PowerShell as Administrator, local network only):

```
New-NetFirewallRule -DisplayName "FastAPI 8000" -Direction Inbound -Protocol TCP -LocalPort 8000 `
  -Action Allow -Profile Private,Public -RemoteAddress LocalSubnet
```

Start the FastAPI server so it listens on the LAN, not only on localhost (inside Factory_Server, venv active):

```
uvicorn main:app --reload --host 0.0.0.0
```

2. Build the firmware

ESP-IDF has no 'rebuild' command. Use build, or a clean build (inside Electronics\Scanner):

```
idf.py build
idf.py fullclean build
```

3. Restore Michael's database dump

Shortcuts for this terminal window (they disappear when it closes):

```
$pg = "C:\Program Files\PostgreSQL\18\bin"
$dump = "C:\Users\sbmit\Downloads\factory.dump"
```

Check that the dump opens (changes nothing):

```
& "$pg\pg_restore.exe" --list $dump | Select-Object -First 15
```

Create the app role and an empty database it owns (asks for the postgres password):

```
& "$pg\psql.exe" -U postgres -p 6900 -c "CREATE USER factory WITH PASSWORD 'CHOOSE_A_PASSWORD';"
& "$pg\psql.exe" -U postgres -p 6900 -c "CREATE DATABASE factory OWNER factory;"
```

Restore the dump into it (asks for the factory password):

```
& "$pg\pg_restore.exe" -U factory -h localhost -p 6900 --no-owner --clean --if-exists -d factory $dump
```

Check the tables arrived:

```
& "$pg\psql.exe" -U factory -h localhost -p 6900 -d factory -c "\dt"
```

4. Run Michael's Django dashboard

Inside Web_Application: create the environment, install packages, apply migrations, start the server:

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
.venv\Scripts\python manage.py migrate
.venv\Scripts\python manage.py runserver 0.0.0.0:8000
```

Optional: make your own admin account.

```
.venv\Scripts\python manage.py createsuperuser
```

Open `http://localhost:8000/admin/` in the browser. `0.0.0.0` is only the listening address and does not work as a URL. The `.env` file next to manage.py holds the DB settings (DB_PORT=6900, DB_NAME=factory, DB_USER=factory) and is git-ignored.

5. Export the schema

Schema only, no data, saved to a file:

```
& "$pg\pg_dump.exe" -U factory -h localhost -p 6900 --schema-only --no-owner `
  -f "$env:USERPROFILE\Downloads\factory_schema.sql" factory
```

Handy check in pgAdmin's Query Tool (database factory):

```
SELECT id, username, is_staff, is_superuser, date_joined FROM auth_user;
```

6. AWS CLI and S3

Install the AWS CLI, then open a NEW terminal so PATH refreshes:

```
winget install Amazon.AWSCLI
aws --version
```

Save your IAM access key on this machine (asks for four values; region is ap-northeast-1, output json):

```
aws configure
```

Check it works (shows your account and user; list buckets):

```
aws sts get-caller-identity
aws s3 ls
```

Make your own bucket (the name must be globally unique; new buckets block public access by default):

```
aws s3 mb s3://YOUR-UNIQUE-BUCKET-NAME --region ap-northeast-1
```

Install the Python tools for the S3 script:

```
python -m pip install boto3 python-dotenv
```

Never commit

`.env` files, `*.dump` files, the AWS access key, or the downloaded `.csv` with its secret. The AWS key lives in `C:\Users\sbmit\.aws\credentials`, outside the project.